

Integração com LLMs — diagnóstico-médico-api

Documentação técnica da camada de explicação clínica via Modelos de Linguagem
Julho de 2026

1. Visão Geral

A **diagnóstico-médico-api** utiliza Modelos de Linguagem de Grande Escala (LLMs) para transformar a saída numérica do modelo de machine learning de triagem de SOP (Síndrome dos Ovários Policísticos) em uma **interpretação clínica estruturada**, escrita em linguagem médica clara e acionável, destinada a médicos especialistas.

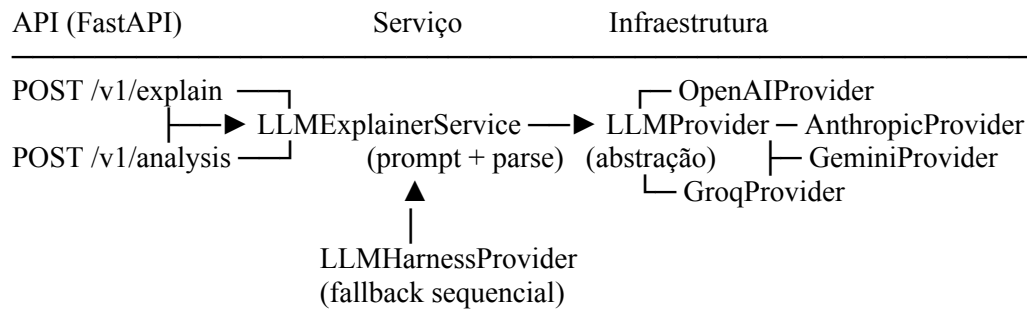
O LLM **não participa do diagnóstico**. A predição (classificação e probabilidade) é feita exclusivamente pelo modelo de ML treinado; o LLM atua apenas como camada de **explicabilidade**, traduzindo o resultado e os fatores determinantes em texto clínico.

1.1. Principais características

Característica	Descrição
Multi-provider	Suporte a OpenAI, Anthropic, Gemini, Groq.
Fallback automático	Cadeia ordenada de provedores (harness) tentados em sequência
Saída estruturada	Resposta em JSON com explicação, fatores de risco e insights
Observabilidade	Logs estruturados (structlog) e eventos no PostHog (sucesso/erro, duração, tokens)
Configuração por ambiente	Todos os provedores e modelos definidos via variáveis de ambiente / .env

2. Arquitetura

A integração segue uma arquitetura em camadas com inversão de dependência: os endpoints dependem de um serviço de domínio (LLMExplainerService), que por sua vez depende de uma abstração (LLMProvider), nunca de um SDK específico.



2.1. Componentes

Componente	Arquivo	Responsabilidade
LLMProvider / LLMResponse	src/app/infrastructure/llm/base.py	Contrato abstrato: generate(system_prompt, user_prompt) -> LLMResponse e provider_name
Provedores concretos	src/app/infrastructure/llm/*_provider.py	Adaptadores para cada SDK/API
create_llm_provider (factory)	src/app/infrastructure/llm/factory.py	Monta o provedor (ou a cadeia) a partir das Settings, valida credenciais
LLMHarnessProvider	src/app/infrastructure/llm/harness.py	Fallback sequencial entre múltiplos provedores
LLMExplainerService	src/app/services/llm_explainer.py	Constrói prompts, chama o provedor, faz o parse do JSON de resposta
Injeção de dependência	src/app/core/dependencies.py	get_llm_provider / get_explainer via Depends do FastAPI
Endpoints	src/app/api/v1/explain/router.py, src/app/api/v1/analysis/router.py	Recebem a requisição, orquestram e registram métricas

2.2. O contrato LLMProvider

Toda a aplicação enxerga os LLMs por meio de uma única interface:

```

@dataclass
class LLMResponse:
    text: str
    tokens_used: int | None = None
  
```

```

class LLMProvider(ABC):
    @abstractmethod
    def generate(self, system_prompt: str, user_prompt: str) -> LLMResponse: ...

    @property
    @abstractmethod
    def provider_name(self) -> str: ...

```

Adicionar um novo provedor exige apenas implementar essa interface e registrá-lo na factory — nenhuma outra camada muda.

3. Provedores Suportados

Provedor	Classe	SDK / Transporte	Modelo padrão	Identificador (provider_name)
OpenAI	OpenAIProvider	openai (Chat Completions)	gpt-4o-mini	openai/<modelo>
Anthropic	AnthropicProvider	anthropic (Messages API)	claude-haiku-4-5-20251001	anthropic/<modelo>
Google Gemini	GeminiProvider	google-genai	gemini-2.5-flash	gemini/<modelo>
Groq	GroqProvider	SDK openai apontando para endpoint OpenAI-compátível do Groq	llama-3.1-8b-instant	groq/<modelo>

Parâmetros de geração são uniformes em todos os provedores: **temperature 0.3** (respostas consistentes e pouco criativas, adequadas ao contexto clínico) e **máximo de 800 tokens de saída**.

Cada provedor extrai a contagem de tokens do campo de *usage* da respectiva API, quando disponível, e a devolve em `LLMResponse.tokens_used` para fins de monitoramen

4. Configuração

Toda a configuração vem de `src/app/core/config.py` (Pydantic Settings), lida do ambiente ou do arquivo `.env`:

Variável	Padrão	Descrição
LLM_PROVIDER	openai	Provedor principal: openai, anthropic, gemini ou groq

Variável	Padrão	Descrição
LLM_FALLBACK_PROVIDERS	(vazio)	Lista ordenada, separada por vírgula, de provedores de fallback (ex.: anthropic,groq)
OPENAI_API_KEY / OPENAI_MODEL	— / gpt-4o-mini	Credencial e modelo OpenAI
ANTHROPIC_API_KEY / ANTHROPIC_MODEL	— / claude-haiku-4-5-20251001	Credencial e modelo Anthropic
GEMINI_API_KEY / GEMINI_MODEL	— / gemini-2.0-flash	Credencial e modelo Gemini
GROQ_API_KEY / GROQ_MODEL	— / llama-3.1-8b-instant	Credencial e modelo Groq

Exemplo de .env com fallback:

```
LLM_PROVIDER=openai
LLM_FALLBACK_PROVIDERS=anthropic,groq
OPENAI_API_KEY=sk-...
ANTHROPIC_API_KEY=sk-ant-...
GROQ_API_KEY=gsk_...
```

A factory valida as credenciais **na criação** de cada provedor da cadeia: se um provedor estiver configurado sem a respectiva API key, é lançada LLMConfigurationError com mensagem indicando a variável de ambiente ausente. Um provedor fora da lista suportada gera ValueError.

5. Cadeia de Fallback (LLM Harness)

Quando LLM_FALLBACK_PROVIDERS está definido, a factory monta um LLMHarnessProvider contendo a cadeia completa (principal + fallbacks, deduplicada preservando a ordem).

Funcionamento do harness a cada chamada generate:

1. Tenta os provedores **em sequência**, na ordem configurada.
2. O primeiro que responder com sucesso encerra a cadeia — a resposta é retornada e o log registra se houve uso de fallback (fallback_used).
3. Qualquer exceção de um provedor é registrada como *warning* (llm_harness_attempt_failed) e a cadeia avança para o próximo.
4. Se **todos** falharem, é lançada LLMFallbackError com o resumo de todas as tentativas e erros.

O provider_name do harness reflete a cadeia, por exemplo: harness[openai/gpt-4o-mini -> anthropic/claude-haiku-4-5-20251001].

6. Fluxo de Geração da Explicação

O LLMExplainerService é o único ponto que constrói prompts e interpreta respostas.

6.1. System prompt

O system prompt define a persona e as regras de segurança clínica:

- Assistente clínico especializado em **endocrinologia reprodutiva e SOP/PCOS**;
- Terminologia médica precisa, em **português do Brasil**;
- Basear-se **exclusivamente nos dados fornecidos** — sem especulação;
- Apresentar o diagnóstico como **probabilidade**, nunca certeza absoluta;
- Ressaltar que o modelo é **ferramenta de triagem**, não substitui avaliação especializada;
- Responder **exclusivamente com JSON válido** no formato:

```
{
  "explanation": "Interpretação clínica em 2-3 parágrafos curtos (máx. 250 palavras)",
  "risk_factors": ["fator de risco 1", "fator de risco 2"],
  "insights": ["recomendação acionável 1", "recomendação acionável 2"]
}
```

6.2. User prompt

Montado dinamicamente a partir da saída do modelo de ML:

- **Diagnóstico:** POSITIVO para SOP ou NEGATIVO para SOP;
- **Probabilidade estimada** (percentual formatado);
- **Dados / fatores determinantes:** lista das features, com nomes traduzidos para forma legível via readable_feature (domínio features.py).

6.3. Execução e parse

1. A chamada ao provedor (síncrona nos SDKs) é despachada para uma thread com `anyio.to_thread.run_sync`, evitando bloquear o event loop do FastAPI.
2. Falha na chamada → `log llm_explanation_failed` e `LLMRequestError` orientando a verificar `LLM_PROVIDER` e as chaves de API.
3. O parse remove cercas de código Markdown (````json`) caso o modelo as inclua, faz `json.loads` e materializa um objeto de domínio `Explanation` (`text`, `risk_factors`, `insights`).
4. Resposta em formato inesperado (JSON inválido ou chave ausente) → `log llm_explanation_parse_failed` e `LLMRequestError` (“resposta em formato inesperado”).

7. Endpoints que Utilizam o LLM

7.1. POST /v1/explain

Recebe o resultado de uma predição já realizada e devolve a interpretação clínica.

Entrada (ExplainInput):

```
{
  "features": {"nome_da_feature": 1.0},
  "diagnosis": 1,
  "probability": 0.87
}
```

Validações: diagnosis $\in \{0, 1\}$; probability $\in [0, 1]$; features sem valores negativos, features binárias restritas a 0/1 e conjunto de features validado por `validate_explain_features`.

Saída (ExplainOutput):

```
{
  "explanation": "...interpretação clínica...",
  "risk_factors": ["..."],
  "insights": ["..."]
}
```

7.2. POST /v1/analysis

Pipeline completo: executa a **predição** (PredictorService) e, na sequência, a **explicação via LLM** com as mesmas features (mapeadas para os nomes originais de coluna esperados pelo `/explain`). Mede a duração da etapa de LLM separadamente para telemetria.

8. Observabilidade e Monitoramento

8.1. Logs estruturados (structlog)

Eventos emitidos ao longo do fluxo, sempre com o provider envolvido:

Evento	Momento
llm_provider_created / llm_harness_created	Criação do provedor/cadeia pela factory
explain_request_received	Chegada da requisição (provider, modelo, diagnosis, probability, features)
llm_explanation_started	Início da geração
llm_harness_attempt_started / _succeeded / _failed	Cada tentativa da cadeia de fallback
llm_explanation_completed	Sucesso (tamanho da resposta, nº de fatores/insights, tokens)
llm_explanation_failed / llm_explanation_parse_failed	Falha de chamada ou de parse
explain_response_sent	Resposta enviada (preview da explicação, tokens)

8.2. Eventos PostHog

- `capture_llm_success(provider, model, duration, tokens_used)` — a cada explicação bem-sucedida;
- `capture_llm_error(provider, model, duration, error)` — a cada falha (a exceção é propagada em seguida).

Habilitado via `POSTHOG_ENABLED=true` e `POSTHOG_API_KEY`.

9. Tratamento de Erros

Exceção	Origem	Situação
LLMConfigurationError	Factory	Provedor configurado sem API key
ValueError	Factory	Provedor não suportado em LLM_PROVIDER
LLMRequestError	LLMExplainerService	Falha na chamada ao provedor ou resposta fora do formato JSON esperado
LLMFallbackError	LLMHarnessProvider	Todos os provedores da cadeia falharam

As mensagens são escritas em português e orientam a ação corretiva (ex.: qual variável de ambiente definir).

10. Como Adicionar um Novo Provedor

1. Criar `src/app/infrastructure/llm/<nome>_provider.py` implementando `LLMProvider` (`generate + provider_name`), retornando `LLMResponse` com `text` e, se disponível, `tokens_used`.
2. Adicionar as settings (`<nome>_api_key`, `<nome>_model`) em `src/app/core/config.py`.
3. Registrar o provedor em `factory.py`: incluir em `_SUPPORTED`, em `_REQUIRED_ENV_VAR` e adicionar o ramo em `_create_single_provider`.
4. Manter os parâmetros de geração alinhados (temperature 0.3, max 800 tokens) salvo justificativa.
5. Cobrir com testes (ver `tests/`) e atualizar esta documentação.

11. Considerações de Segurança e Uso Clínico

- O LLM atua somente como camada de explicação; **a decisão diagnóstica vem do modelo de ML** e é sempre apresentada como probabilidade.
- O system prompt impõe explicitamente que o resultado é **triagem** e não substitui avaliação médica especializada.

- Nenhum dado identificável do paciente é enviado ao provedor — apenas features clínicas numéricas/binárias e o resultado do modelo.
- As chaves de API nunca são versionadas: residem em variáveis de ambiente / .env (e Secrets Manager na infraestrutura).